

Introduction to Computer Systems Design

CS3031 · IIT Madras · Week 3

Module 1 → 2: Minimizing Combinational Logic

Detailed Study Notes

Canonical Forms, Universal Gates and Minimising Logic

- ▶ Minterm numbering and the $\sum m(\dots)$ shorthand
- ▶ Canonical SOP $\neq$ minimal: deriving $F = A + BC$
- ▶ Two-level AND–OR circuits, gate levels and cost
- ▶ Product of Sums (POS), maxterms and $\prod M(\dots)$
- ▶ The four minterm/maxterm conversions, with proofs
- ▶ Universal gates: NAND–NAND and NOR–NOR realisations
- ▶ Multilevel logic: area vs delay
- ▶ Timing diagrams, don't-cares, and the tutorial problems (circuit optimisation; the three-switch bulb)
- ▶ 30 numericals, 20 subjective Qs, 40 MCQs

Based on lecture slides by Ayon Chakraborty, Department of CSE, IIT Madras. Boxes marked *Beyond the Slides* add standard textbook context that is not on the slides. Handwritten annotations on the slides have been interpreted and typeset; where they were ambiguous, this is flagged.

Contents

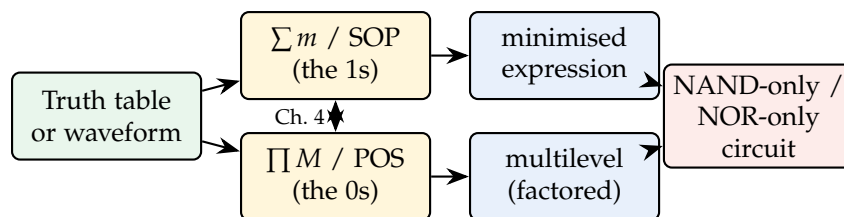
How to Use These Notes	3
I Module 1 → 2: Minimising Combinational Logic	4
1 Minterm Notation and Canonical SOP	5
1.1 Numbering the rows	5
1.2 Canonical SOP forms	6
1.2.1 Deriving the minimal form step by step	6
1.2.2 Cost of canonical vs minimal	7
2 From Logic Gates to Circuits: Two-Level Logic	8
2.1 Why “two-level”?	8
2.2 The example, simplified	9
3 The Alternative Canonical Form: Product of Sums	10
3.1 Idea	10
3.2 Derivation: POS is De Morgan of the SOP of $\bar{F}$	10
3.3 Why POS works: checking one row	11
3.3.1 General proof	11
3.4 How to write the POS	11
3.5 Canonical POS	12
3.6 POS → OR–AND circuit, and minimising a POS	12
4 Useful Conversions between $\sum m$ and $\prod M$	14
4.1 The one picture behind all four rules	14
4.2 Proofs	14
4.3 Worked conversions	15
5 Universal Gates: NAND and NOR	16
5.1 The NAND and NOR gates	16
5.2 NAND is universal	16
5.3 AND–OR → NAND–NAND	17
5.4 NOR is universal	17
6 Multilevel Logic	19
6.1 Comparing with two-level forms	19

7	Aside: Timing Diagrams	21
7.1	Reading a timing diagram	21
7.2	Don't-care conditions	22
7.3	Beyond the slides: what real timing diagrams show	22
8	Tutorial and Practice Problems from the Slides	23
8.1	Tutorial 1: optimise this circuit	23
8.1.1	Solution	23
8.2	Tutorial 2: the three-switch bulb	24
8.2.1	Step 1: the specification	24
8.2.2	Step 2: the logic	24
8.2.3	Step 3: physical implementation	25
II	Practice	26
9	Numerical Problems with Worked Solutions	27
9.1	Minterm and maxterm notation	27
9.2	Conversions	28
9.3	Minimisation	28
9.4	Universal gates	30
9.5	Multilevel logic and timing	31
9.6	The switch problem	32
10	Subjective Questions with Answers	33
11	Multiple-Choice Questions	37
11.1	Questions	37
11.2	Answers and Explanations	43
A	Formula Sheet, Glossary and Revision	45
A.1	Formula sheet	45
A.2	Glossary	45
A.3	Revision checklist	46

How to Use These Notes

Week 2 ended with a way to turn *any* truth table into a circuit: canonical Sum of Products. Week 3 asks the engineer's follow-up questions:

- Is there a compact notation for canonical forms? ($\sum m, \prod M$)
- Is there a second, "dual" canonical form? (Product of Sums)
- Can the circuit be smaller? (canonical $\neq$ minimal; multilevel logic)
- Can it be built from a single gate type? (NAND and NOR are universal)
- How do we read behaviour over time? (timing diagrams)



The coloured boxes

Intuition

The mental picture behind an idea.

Key Idea

The one sentence to remember.

Formula

A result with its derivation; sizes/counts in grey brackets like [rows] or [gates].

Common Pitfall

A common exam mistake.

Beyond the Slides

Standard background not on the slides (Mano; Harris & Harris).

Notation

m_i	minterm i : product that is 1 only in row i
M_i	maxterm i : sum that is 0 only in row i
$\sum m(i, j, \dots)$	OR of the listed minterms (canonical SOP)
$\prod M(i, j, \dots)$	AND of the listed maxterms (canonical POS)
$d(\dots)$ or X	don't-care rows (output may be 0 or 1)
A is the most significant bit	row number = $4A + 2B + C$ for three variables

Part I

Module 1 $\rightarrow$ 2: Minimising Combinational Logic

Chapter 1

Minterm Notation and Canonical SOP

1.1 Numbering the rows

From the slides:

Notation for SOP Standard “shorthand” notation: if we agree on the order of the variables in the rows of the truth table, then we can enumerate each row with the decimal number that corresponds to the binary number created by the input pattern. For A, B, C with $F = 0, 0, 0, 1, 1, 1, 1$: 100 = decimal 4, so this is minterm #4, or m_4 ; 111 = decimal 7, so this is minterm #7, or m_7 . $f = m_3 + m_4 + m_5 + m_6 + m_7 = \sum m(3, 4, 5, 6, 7)$ — we can write this as a sum of products, or use a summation notation. (Handwritten: “(1) # of variables, (2) order”.)

Formula : row (minterm) index

With variable order A, B, C (A = most significant bit):

$$i = 4A + 2B + C = A \cdot 2^2 + B \cdot 2^1 + C \cdot 2^0 \quad [\text{row number, } 0 \leq i \leq 2^n - 1]$$

In general, for n variables $x_{n-1} \dots x_0$: $i = \sum_k x_k 2^k$ — positional notation from Week 1.

row i	A	B	C	minterm m_i	F
0	0	0	0	$\overline{A} \overline{B} \overline{C}$	0
1	0	0	1	$\overline{A} \overline{B} C$	0
2	0	1	0	$\overline{A} B \overline{C}$	0
3	0	1	1	$\overline{A} B C$	1
4	1	0	0	$A \overline{B} \overline{C}$	1
5	1	0	1	$A \overline{B} C$	1
6	1	1	0	$A B \overline{C}$	1
7	1	1	1	$A B C$	1

$$\begin{aligned} F(A, B, C) &= \sum m(3, 4, 5, 6, 7) \\ &= m_3 + m_4 + m_5 + m_6 + m_7 \\ &= \overline{A} B C + A \overline{B} \overline{C} + A \overline{B} C \\ &\quad + A B \overline{C} + A B C \end{aligned}$$

Common Pitfall : the order must be stated

$\sum m(3, 4, 5, 6, 7)$ means nothing until you say which variable is the MSB. Write $F(A, B, C)$, not just F . With the order reversed to $F(C, B, A)$, row $ABC = 100$ would be number $1 \cdot 1 + 0 \cdot 2 + 0 \cdot 4 = 1$, not 4 — a completely different list. The handwritten notes on the slide stress exactly this: (1) the number of variables, (2) their order.

Key Idea

The shorthand is lossless: from $F(A, B, C) = \sum m(3, 4, 5, 6, 7)$ you can rebuild the whole truth table (the listed rows are 1, all other rows are 0). It is the truth table’s output column written as a set of row numbers.

1.2 Canonical SOP forms

From the slides:

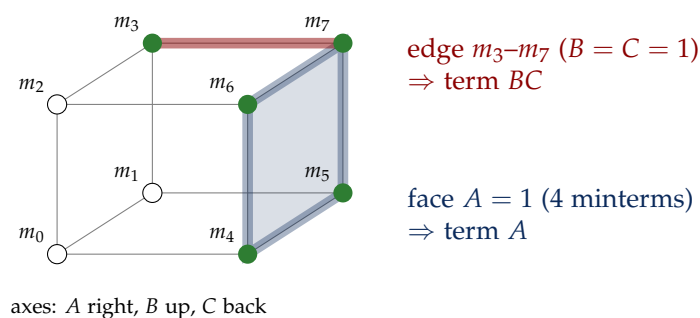
Canonical SOP Forms Table of all eight 3-variable minterms, $\overline{A}\overline{B}\overline{C} = m_0$ through $ABC = m_7$ ("Shorthand notation for minterms of 3 variables"). F in canonical form: $F(A,B,C) = \sum m(3,4,5,6,7) = m_3 + m_4 + m_5 + m_6 + m_7$, $F = \overline{A}BC + \overline{A}\overline{B}C + \overline{A}BC + A\overline{B}C + ABC$. **Canonical form $\neq$ minimal form** (handwritten: "VERY IMP"): $F = \overline{A}B(\overline{C} + C) + \overline{A}BC + AB(\overline{C} + C) = \overline{A}B + \overline{A}BC + AB = A(\overline{B} + B) + \overline{A}BC = A + \overline{A}BC = A + BC$. 2-level AND/OR realisation.

1.2.1 Deriving the minimal form step by step

$$\begin{aligned}
 F &= \overline{A}BC + \overline{A}\overline{B}C + \overline{A}BC + A\overline{B}C + ABC \\
 &= \overline{A}B(\overline{C} + C) + \overline{A}BC + AB(\overline{C} + C) && \text{group pairs differing only in } C \text{ (A5)} \\
 &= \overline{A}B + \overline{A}BC + AB && A6, A4 \\
 &= A(\overline{B} + B) + \overline{A}BC && \text{group again (A5)} \\
 &= A + \overline{A}BC && A6, A4 \\
 &= A + BC && \text{T11D: } X + \overline{X}Y = X + Y \text{ with } X = A, Y = BC
 \end{aligned}$$

Intuition : why the pairing works

Two minterms that differ in exactly one variable (like $\overline{A}\overline{B}\overline{C}$ and $\overline{A}\overline{B}C$) are "neighbours". Together they say " $\overline{A}\overline{B}$, and C can be anything", so C disappears. Each successful pairing halves the number of rows a term must name and removes one literal. Pairing neighbours systematically is exactly what Karnaugh maps do (a standard next step in Mano, Ch. 3).

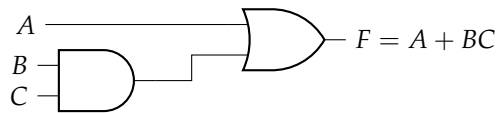


Each minterm is a corner of a cube. Neighbours differ in one variable. The four 1-corners on the face $A = 1$ merge into the single literal A ; the edge m_3 – m_7 merges into BC . m_7 is used twice — allowed, because $X + X = X$ (idempotence).

Beyond the Slides : using idempotence to re-use a minterm

The slide's grouping uses each minterm once. Another route: duplicate $m_7 = ABC$ (legal since $X = X + X$) and pair it with m_3 : $\overline{A}BC + ABC = BC$; and the four minterms m_4, m_5, m_6, m_7 together give A . Same answer, $F = A + BC$, fewer steps. Re-using a 1 in several groups is the key trick of K-map minimisation.

1.2.2 Cost of canonical vs minimal



Form	gates	gate inputs	inverters
Canonical SOP $\sum m(3, 4, 5, 6, 7)$	5 AND3 + 1 OR5	20	3
Minimal $A + BC$	1 AND2 + 1 OR2	4	0

Same function, five times fewer gate inputs.

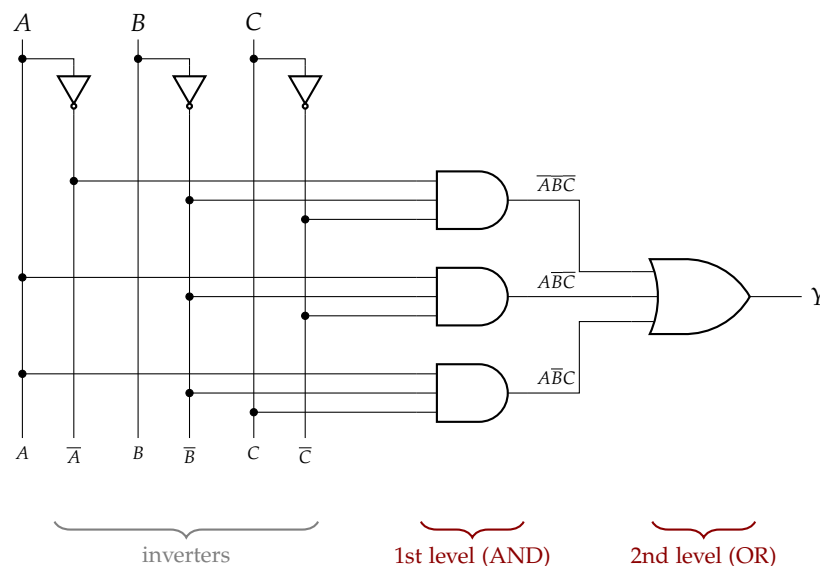
Key Idea

Canonical $\neq$ **minimal** (“VERY IMP” on the slide). Canonical forms are unique and mechanical — ideal for specifying and comparing functions. Minimal forms are what you build. Getting from one to the other is the job of Boolean algebra (and, later, systematic methods like Karnaugh maps).

From Logic Gates to Circuits: Two-Level Logic

From the slides:

From Logic Gates to Circuits SOP (sum-of-products) leads to **two-level logic**. Example: $Y = (\overline{A} \cdot \overline{B} \cdot \overline{C}) + (A \cdot \overline{B} \cdot \overline{C}) + (A \cdot \overline{B} \cdot C)$. Circuit: vertical input lines A, B, C , each with an inverter producing $\overline{A}, \overline{B}, \overline{C}$; three 3-input AND gates (minterms $\overline{A}\overline{B}\overline{C}$, $A\overline{B}\overline{C}$, $A\overline{B}C$) feeding one 3-input OR giving Y . Handwritten: “1st level” at the ANDs, “2nd level” at the OR, and delay marks.



2.1 Why “two-level”?

Count the gates any signal passes through from input to output, ignoring the inverters that make complemented inputs available: one AND, then one OR. Every SOP expression has this shape — a layer of ANDs feeding a single OR — and every POS expression (Chapter 3) has the mirror shape, ORs feeding an AND.

Formula : delay of two-level logic

If each gate has propagation delay t_{pd} [s], the worst-case input-to-output delay of an SOP circuit is

$$t_{\text{SOP}} = t_{\text{NOT}} + t_{\text{AND}} + t_{\text{OR}} \approx 3t_{pd} \quad (\text{or } 2t_{pd} \text{ if complemented inputs are already available}).$$

It is fixed, no matter how complicated the function. That is the big attraction of two-level logic: **predictable, small delay**. Its weakness is size (Chapter 6).

2.2 The example, simplified

row	A	B	C	Y
0	0	0	0	1
1	0	0	1	0
2	0	1	0	0
3	0	1	1	0
4	1	0	0	1
5	1	0	1	1
6	1	1	0	0
7	1	1	1	0

$$Y(A, B, C) = \sum m(0, 4, 5)$$

$$\begin{aligned}
 Y &= \overline{A}\overline{B}\overline{C} + \overline{A}\overline{B}C + \overline{A}B\overline{C} \\
 &= \overline{B}\overline{C}(\overline{A} + A) + \overline{A}B\overline{C} \\
 &= \overline{B}\overline{C} + \overline{A}B\overline{C} \\
 &= \overline{B}(\overline{C} + AC) \\
 &= \overline{B}(\overline{C} + A) \quad (\text{T11D}) \\
 &= \overline{B}\overline{C} + A\overline{B}
 \end{aligned}$$

Three 3-input ANDs and a 3-input OR become one 2-input OR and one 2-input AND ($\overline{B}(A + \overline{C})$), or two 2-input ANDs and an OR ($\overline{B}\overline{C} + A\overline{B}$).

Common Pitfall

When pairing terms, you may reuse a minterm (e.g. m_4 pairs with m_0 to give $\overline{B}\overline{C}$ and with m_5 to give $A\overline{B}$), but you must not drop one: every 1 of the function must be covered by at least one term, and no term may cover a 0.

The Alternative Canonical Form: Product of Sums

3.1 Idea

From the slides:

Alternative Canonical Form: POS We can have another form of representation: **DeMorgan of SOP of $\bar{F}$** . A product of sums (POS): $F = (A + B + C)(A + B + \bar{C})(A + \bar{B} + C)$. Each sum term represents one of the “zeros” of the function. For the given input (010), only the shaded sum term will equal 0: $A + \bar{B} + C = 0 + 1 + 0$. Anything ANDed with 0 is 0; output F will be 0.

SOP describes a function by listing where it is **1**. POS describes it by listing where it is **0**: “ F is 1 unless the input is one of these forbidden rows.”

Intuition : “allowed” list vs “banned” list

A club can publish a guest list (only these people get in: SOP) or a banned list (everyone gets in except these: POS). If the club lets almost everyone in, the banned list is shorter. Likewise, a function with few 0s has a short POS and a function with few 1s has a short SOP.

3.2 Derivation: POS is De Morgan of the SOP of $\bar{F}$

For our running function $F(A, B, C) = \sum m(3, 4, 5, 6, 7)$, the complement $\bar{F}$ is 1 exactly in the rows where $F = 0$: rows 0, 1, 2. So by Week 2’s SOP recipe,

$$\bar{F} = m_0 + m_1 + m_2 = \bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}C + \bar{A}B\bar{C}.$$

Complement both sides and apply De Morgan twice (first across the +, then inside each product):

$$\begin{aligned} F = \bar{\bar{F}} &= \overline{\bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}C + \bar{A}B\bar{C}} \\ &= \overline{\bar{A}\bar{B}\bar{C}} \cdot \overline{\bar{A}\bar{B}C} \cdot \overline{\bar{A}B\bar{C}} && \text{De Morgan on the sum} \\ &= (A + B + C)(A + B + \bar{C})(A + \bar{B} + C) && \text{De Morgan on each product, involution} \end{aligned}$$

Formula : minterm and maxterm are complements

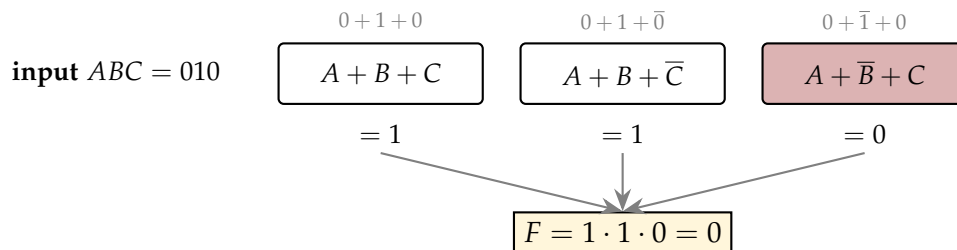
$$M_i = \overline{m_i} \quad m_i = \overline{M_i}$$

E.g. $m_2 = \bar{A}B\bar{C}$, $M_2 = \overline{\bar{A}B\bar{C}} = A + \bar{B} + C$.

3.3 Why POS works: checking one row

From the slides:

Consider $A=0, B=1, C=0$ $F = (A + B + C)(A + B + \bar{C})(A + \bar{B} + C)$ with 010 substituted: $(0 + 1 + 0) = 1, (0 + 1 + \bar{0}) = 1, (0 + \bar{1} + 0) = 0$. Only one of the products will be 0; anything ANDed with 0 is 0. Therefore, the output is $F = 0$. (Handwritten: "SOP expression and POS expression just the entries in the truth table"; "POS/SOP $\rightarrow$ canonical expression, NOT NECESSARILY the minimal expression".)



3.3.1 General proof

Each maxterm M_i is 0 in exactly its own row i and 1 everywhere else (dual of the minterm property).

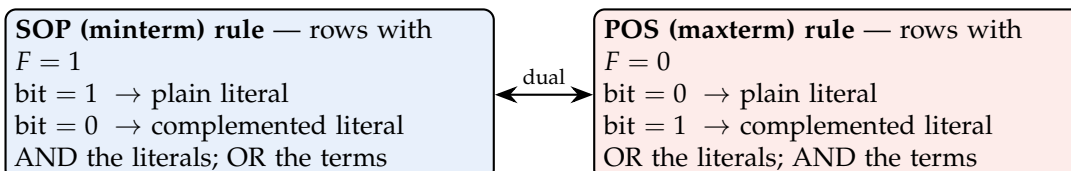
- In a row where $F = 0$, that row's maxterm is in the product and equals 0, so the AND gives 0.
- In a row where $F = 1$, no maxterm in the product belongs to that row, so all are 1 and the AND gives 1.

Hence canonical POS always equals F : every Boolean function can be written in POS form.

3.4 How to write the POS

From the slides:

POS – How to Write? Maxterm form: 1. Find truth table rows where F is 0. 2. 0 in input column $\rightarrow$ true literal. 3. 1 in input column $\rightarrow$ complemented literal. 4. OR the literals to get a maxterm. 5. AND together all the maxterms. Or just remember, POS of F is the same as the DeMorgan of SOP of $\bar{F}$!



Worked Example : row $ABC = 010$

$A = 0 \rightarrow A, B = 1 \rightarrow \bar{B}, C = 0 \rightarrow C$: maxterm $A + \bar{B} + C = M_2$. It is 0 exactly when $A = 0, B = 1, C = 0$ — i.e. only in row 2.

Common Pitfall

The literal rules are *opposite* for minterms and maxterms. For row 010 the minterm is $\overline{A}B\overline{C}$ but the maxterm is $A + \overline{B} + C$. Mixing these up is the most common POS mistake.

3.5 Canonical POS**From the slides:**

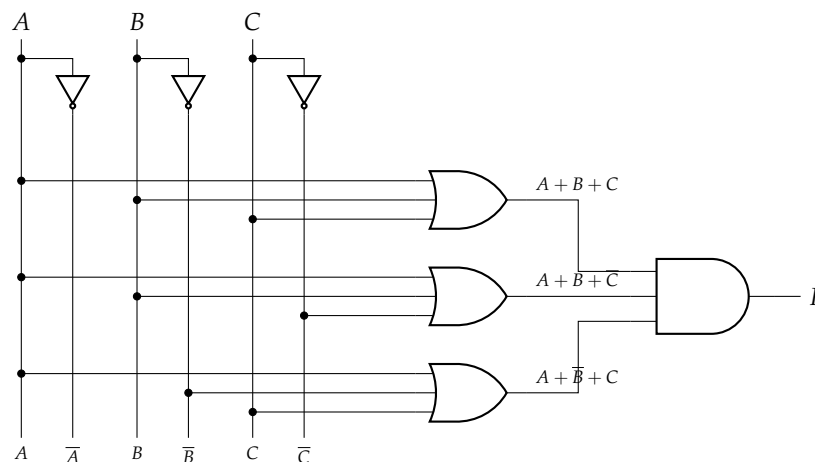
Canonical POS Product of Sums / Conjunctive Normal Form / Maxterm Expansion. $F = (A + B + C)(A + B + \overline{C})(A + \overline{B} + C) = \prod M(0, 1, 2)$. Maxterm shorthand notation for a function of three variables: $A + B + C = M_0$, $A + B + \overline{C} = M_1$, $A + \overline{B} + C = M_2$, $A + \overline{B} + \overline{C} = M_3$, $\overline{A} + B + C = M_4$, $\overline{A} + B + \overline{C} = M_5$, $\overline{A} + \overline{B} + C = M_6$, $\overline{A} + \overline{B} + \overline{C} = M_7$. Note that you form the maxterms around the “zeros” of the function. **This is not the complement of the function!**

i	A	B	C	minterm m_i (1 only here)	maxterm M_i (0 only here)	F
0	0	0	0	$\overline{A}\overline{B}\overline{C}$	$A + B + C$	0
1	0	0	1	$\overline{A}\overline{B}C$	$A + B + \overline{C}$	0
2	0	1	0	$\overline{A}B\overline{C}$	$A + \overline{B} + C$	0
3	0	1	1	$\overline{A}BC$	$A + \overline{B} + \overline{C}$	1
4	1	0	0	$A\overline{B}\overline{C}$	$\overline{A} + B + C$	1
5	1	0	1	$A\overline{B}C$	$\overline{A} + B + \overline{C}$	1
6	1	1	0	$AB\overline{C}$	$\overline{A} + \overline{B} + C$	1
7	1	1	1	ABC	$\overline{A} + \overline{B} + \overline{C}$	1

$$F(A, B, C) = \prod M(0, 1, 2) = M_0 M_1 M_2 = (A + B + C)(A + B + \overline{C})(A + \overline{B} + C).$$

Common Pitfall : “not the complement”

$\prod M(0, 1, 2)$ is F itself, built around F 's zeros. It is *not* $\overline{F}$. What is true is: $\overline{F} = \sum m(0, 1, 2)$ — the same index list, but with minterms. Remember: same indices + switch $\sum \leftrightarrow \prod$ = complement; complementary indices + switch = same function (Chapter 4).

3.6 POS $\rightarrow$ OR–AND circuit, and minimising a POS

Two-level OR–AND: first level ORs (one per maxterm), second level one AND.

POS can be simplified with the dual theorems:

$$\begin{aligned}
 F &= (A + B + C)(A + B + \overline{C})(A + \overline{B} + C) \\
 &= (A + B + C)(A + B + \overline{C}) \cdot (A + B + C)(A + \overline{B} + C) && \text{T3D: reuse } M_0 \\
 &= (A + B) \cdot (A + C) && \text{T9D: } (X + Y)(X + \overline{Y}) = X \text{ twice} \\
 &= A + BC && \text{A5 (OR distributes over AND)}
 \end{aligned}$$

The SOP and POS routes arrive at the same minimal function, as they must.

Key Idea

For every function: canonical SOP lists the 1s as minterms, canonical POS lists the 0s as maxterms. Both are unique and both are “just the entries of the truth table” (handwritten note) — neither is necessarily minimal. Choose SOP when the function has few 1s and POS when it has few 0s; for $\sum m(3, 4, 5, 6, 7)$, POS has 3 terms against SOP’s 5.

Chapter 4

Useful Conversions between $\sum m$ and $\prod M$

From the slides:

Useful Conversions 1. **Minterm to Maxterm conversion:** rewrite minterm shorthand using maxterm shorthand; replace minterm indices with the indices not already used. E.g. $F(A, B, C) = \sum m(3, 4, 5, 6, 7) = \prod M(0, 1, 2)$. 2. **Maxterm to Minterm conversion:** rewrite maxterm shorthand using minterm shorthand; replace indices with those not already used. E.g. $\prod M(0, 1, 2) = \sum m(3, 4, 5, 6, 7)$. 3. **Expansion of F to expansion of $\bar{F}$:** $F = \sum m(3, 4, 5, 6, 7) = \prod M(0, 1, 2) \rightarrow \bar{F} = \sum m(0, 1, 2) = \prod M(3, 4, 5, 6, 7)$. 4. **Minterm expansion of F to maxterm expansion of $\bar{F}$:** rewrite in maxterm form using the same indices as F : $F = \sum m(3, 4, 5, 6, 7) \rightarrow \bar{F} = \prod M(3, 4, 5, 6, 7) = \sum m(0, 1, 2)$. (Handwritten: “ $\sum m$ = sum (SOP)”, “ $\prod M$ = product (POS)”, “entries where value of $F = 1$ ”, “value of $F = 0$ ”.)

4.1 The one picture behind all four rules

Split the 2^n row indices into S (rows where $F = 1$) and its complement S' (rows where $F = 0$).

$S = \{3, 4, 5, 6, 7\}$ rows where $F = 1$	$S' = \{0, 1, 2\}$ rows where $F = 0$
---	--

$$F = \sum m(S) = \prod M(S')$$

$$\bar{F} = \sum m(S') = \prod M(S)$$

#	Start from	Want	Rule
1	$F = \sum m(S)$	F as $\prod M$	$\prod M(S')$ (swap symbol, complementary indices)
2	$F = \prod M(S')$	F as $\sum m$	$\sum m(S)$ (swap symbol, complementary indices)
3	$F = \sum m(S)$	$\bar{F}$ as $\sum m$	$\sum m(S')$ (same symbol, complementary indices)
4	$F = \sum m(S)$	$\bar{F}$ as $\prod M$	$\prod M(S)$ (swap symbol, same indices)

Key Idea

Same function: switch $\sum \leftrightarrow \prod$ and switch to the missing indices. **Complement:** change exactly one of the two (symbol or index set).

4.2 Proofs

Rule 1 and 2. $\sum m(S)$ is 1 exactly on S (Week 2). $\prod M(S')$ is 0 exactly on S' , i.e. 1 exactly on S (Chapter 3). Same truth table $\Rightarrow$ same function.

Rule 3. $\bar{F}$ is 1 exactly where F is 0, i.e. on S' , so $\bar{F} = \sum m(S')$ by the SOP recipe.

Rule 4. Complement $F = \sum_{i \in S} m_i$ with De Morgan: $\bar{F} = \prod_{i \in S} \bar{m}_i = \prod_{i \in S} M_i = \prod M(S)$, since $\bar{\bar{m}_i} = M_i$.

4.3 Worked conversions

Worked Example : four variables

$G(A, B, C, D) = \sum m(0, 2, 5, 7, 8, 10, 13, 15)$. With $2^4 = 16$ rows, $S' = \{1, 3, 4, 6, 9, 11, 12, 14\}$.

- Rule 1: $G = \prod M(1, 3, 4, 6, 9, 11, 12, 14)$.
- Rule 3: $\overline{G} = \sum m(1, 3, 4, 6, 9, 11, 12, 14)$.
- Rule 4: $\overline{G} = \prod M(0, 2, 5, 7, 8, 10, 13, 15)$.

(Incidentally, $G = \overline{B}\overline{D} + BD$: the 1s are exactly the rows with $B = D$.)

Common Pitfall

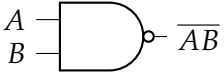
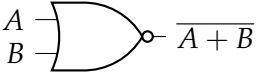
Complementary indices are taken over *all* 2^n rows. For a 4-variable function that is 0–15, not 0–7. Always write down n first.

Chapter 5

Universal Gates: NAND and NOR

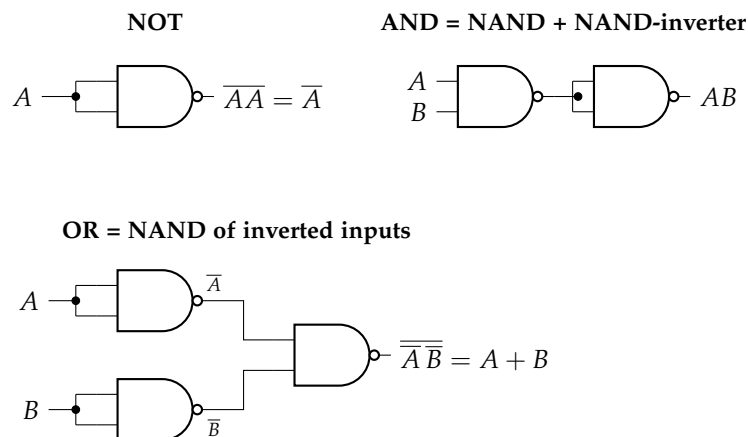
A set of gates is **universal** (functionally complete) if every Boolean function can be built from it. Week 2 showed that {AND, OR, NOT} is universal (every function has an SOP). This chapter shows that NAND alone, and NOR alone, are universal too.

5.1 The NAND and NOR gates

					
A	B	NAND	A	B	NOR
0	0	1	0	0	1
0	1	1	0	1	0
1	0	1	1	0	0
1	1	0	1	1	0

NAND = “NOT AND” (0 only when all inputs are 1). NOR = “NOT OR” (1 only when all inputs are 0). These are the handwritten “NAND: not AND”, “NOR: not OR” notes on the slides.

5.2 NAND is universal



Formula : NAND realisations

NOT: $\overline{A \cdot A} = \overline{A}$

(idempotence $AA = A$)

AND: $\overline{\overline{A} \overline{B}} = AB$

(involution)

OR: $\overline{\overline{A} \cdot \overline{B}} = \overline{\overline{A}} + \overline{\overline{B}} = A + B$

(De Morgan, involution)

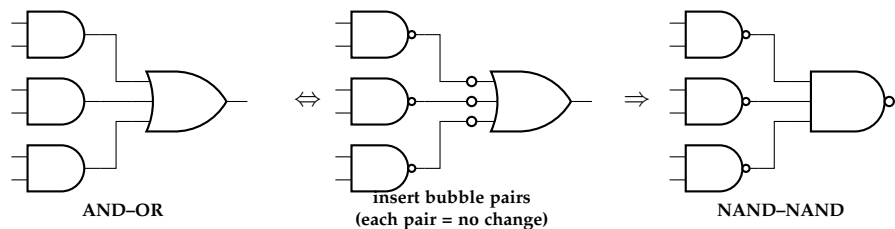
Gate counts: NOT = 1, AND = 2, OR = 3 NANDs.

Since NOT, AND, OR can each be made of NANDs, and {NOT, AND, OR} is universal, **NAND is universal**. (A NAND input can also be inverted by tying the other input to 1: $\overline{A} \cdot 1 = \overline{A}$.)

5.3 AND-OR $\rightarrow$ NAND-NAND

From the slides:

Universal Gates: NAND Three stages: **AND-OR** (three ANDs feeding an OR, handwritten “SOP”) $\Leftrightarrow$ the same circuit with pairs of inverters inserted on every internal wire (one inverter absorbed into each AND making a NAND, the other absorbed into the OR’s inputs making an OR-with-bubbled-inputs = NAND) $\Rightarrow$ **NAND-NAND**. Handwritten: “ $\overline{a} \cdot \overline{b} = \overline{a + b}$ DeMorgan’s law”; “only one gate: NAND”.



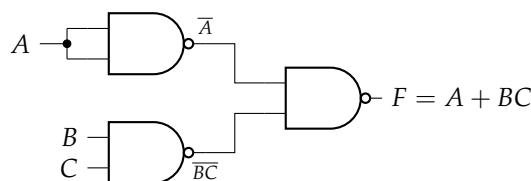
Derivation. Start from $F = P_1 + P_2 + P_3$ with P_k products. Apply involution then De Morgan:

$$F = \overline{\overline{P_1 + P_2 + P_3}} = \overline{\overline{P_1} \cdot \overline{P_2} \cdot \overline{P_3}} = \text{NAND}(\text{NAND}(\text{inputs of } P_1), \text{NAND}(\dots), \text{NAND}(\dots)).$$

Rule: any SOP becomes a two-level NAND-NAND circuit by replacing every gate with a NAND — same wiring. (A single-literal term, like A in $A + BC$, enters the output NAND complemented: $\overline{A}$.)

Worked Example : $F = A + BC$ with NANDs only

$F = \overline{\overline{A + BC}} = \overline{\overline{A} \cdot \overline{BC}} = \text{NAND}(\overline{A}, \text{NAND}(B, C))$, and $\overline{A} = \text{NAND}(A, A)$. Total: 3 two-input NANDs.



5.4 NOR is universal

Formula : NOR realisations

$$\text{NOT: } \overline{A + A} = \overline{A}$$

$$\text{OR: } \overline{\overline{A + B}} = A + B$$

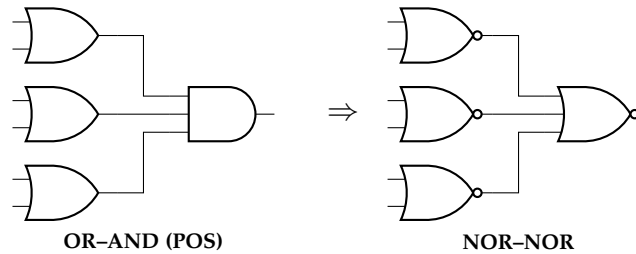
$$\text{AND: } \overline{\overline{A} + \overline{B}} = AB$$

(De Morgan)

Gate counts: NOT = 1, OR = 2, AND = 3 NORs.

From the slides:

Universal Gates: NOR **OR-AND** (three ORs feeding an AND, handwritten “POS”) $\Leftrightarrow$ bubble pairs inserted $\Rightarrow$ **NOR-NOR**. Handwritten: “ $(x + y)'$ NOR”; “TT $\rightarrow$ POS/SOP $\rightarrow$ implementable using only NAND or only NOR gates”.



Derivation. $F = S_1 S_2 S_3$ (sums). $F = \overline{\overline{S_1 S_2 S_3}} = \overline{\overline{S_1} + \overline{S_2} + \overline{S_3}} = \text{NOR}(\text{NOR}(\dots), \text{NOR}(\dots), \text{NOR}(\dots))$.

Worked Example : $F = (A + B)(A + C)$ with NORs only

$F = \text{NOR}(\text{NOR}(A, B), \text{NOR}(A, C))$. Check $A = 0, B = 1, C = 0$: $\text{NOR}(0, 1) = 0$, $\text{NOR}(0, 0) = 1$, $F = \text{NOR}(0, 1) = 0$; and $(0 + 1)(0 + 0) = 0 \checkmark$.

Key Idea

Truth table $\rightarrow$ SOP $\rightarrow$ NAND–NAND, or truth table $\rightarrow$ POS $\rightarrow$ NOR–NOR. Any function can be built from a *single* gate type (handwritten: “only one gate”). Manufacturing needs only one well-optimised cell.

Beyond the Slides : why NAND/NOR rather than AND/OR in CMOS

In CMOS, a gate is naturally *inverting*: an input high turns on nMOS transistors that pull the output low (Week 1’s inverter). A 2-input NAND needs 4 transistors; a 2-input AND is a NAND followed by an inverter, 6 transistors. So NAND and NOR are the cheapest and fastest gates, and real designs are built mostly from them. NAND is usually preferred over NOR because its series transistors are the faster nMOS type.

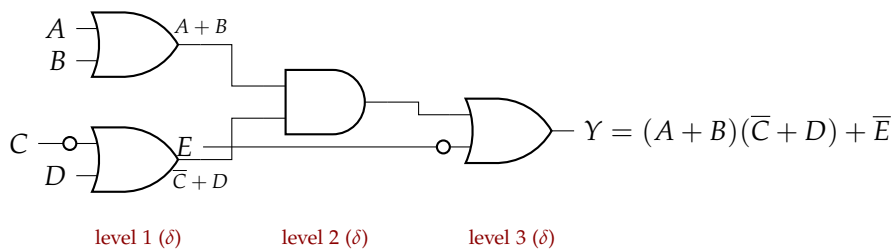
Common Pitfall

NAND is not associative: $\text{NAND}(\text{NAND}(A, B), C) \neq \text{NAND}(A, \text{NAND}(B, C))$ in general (try $A = 0, B = 0, C = 1$: left = $\text{NAND}(1, 1) = 0$, right = $\text{NAND}(0, 1) = 1$). So a 3-input NAND is *not* two 2-input NANDs in a chain; it is $\overline{ABC}$.

Multilevel Logic

From the slides:

Multilevel Logic So far, we have primarily focused on two-level representations for combinational logic. Multi-level logic is typically more compact (i.e., cost-efficient) in practice. Circuit: A, B into an OR; $\bar{C}$ (bubbled input) and D into an OR; the two ORs into an AND; the AND and $\bar{E}$ (bubbled input) into a final OR: $Y = (A + B)(\bar{C} + D) + \bar{E}$. (Handwritten: a delay “ δ ” marked on each gate level, and “ $(A + B)(\bar{C} + D)$ ”, “extra term”.)



6.1 Comparing with two-level forms

Expand the product to get the two-level SOP:

$$Y = (A + B)(\bar{C} + D) + \bar{E} = A\bar{C} + AD + B\bar{C} + BD + \bar{E}.$$

Alternatively, distribute the final OR over the AND (the “unusual” distributive law, $X + YZ = (X + Y)(X + Z)$):

$$Y = ((A + B) + \bar{E})(\bar{C} + D + \bar{E}) = (A + B + \bar{E})(\bar{C} + D + \bar{E}),$$

a two-level POS.

Realisation	gates	gate inputs	levels (delay)	literals
Multilevel $(A + B)(\bar{C} + D) + \bar{E}$	4	8	3 (3δ)	5
Two-level SOP $A\bar{C} + AD + B\bar{C} + BD + \bar{E}$	5 (4 AND2 + OR5)	13	2 (2δ)	9
Two-level POS $(A + B + \bar{E})(\bar{C} + D + \bar{E})$	3 (2 OR3 + AND2)	8	2 (2δ)	6

Inverters for $\bar{C}$ and $\bar{E}$ are needed in every version and are not counted.

Intuition : factoring is sharing

In $A\bar{C} + AD + B\bar{C} + BD$ each of $A, B, \bar{C}, D$ is wired to two gates. Factoring into $(A + B)(\bar{C} + D)$ computes “ A or B ” once and re-uses it — like writing $ac + ad + bc + bd = (a + b)(c + d)$ in ordinary algebra. Shared sub-expressions are what make multilevel logic smaller.

Key Idea : the area–delay trade-off

Two-level logic has the fewest gate levels, so the smallest, most predictable delay, but can need many gates and wide fan-in. Multilevel logic is usually more compact but adds levels, each costing one gate delay δ (the handwritten δ s on the slide). Real synthesis tools search this trade-off automatically. (In this particular example, the POS happens to win on both counts — a reminder to try both canonical routes.)

Formula : delay of a path

$$t_{\text{path}} = \sum_{\text{gates on path}} \delta_{\text{gate}} \text{ [s]}; \quad t_{\text{circuit}} = \max_{\text{paths}} t_{\text{path}} \quad (\text{the critical path})$$

For the multilevel circuit, the paths from A, B, C, D go through three gates (3δ); the path from E goes through only one (δ). The circuit's delay is set by the slowest path: 3δ .

Beyond the Slides : fan-in

A gate's number of inputs is its *fan-in*. Large fan-in gates (like the 5-input OR of the SOP) are slow in CMOS because they need long chains of series transistors, so real libraries offer only up to 4 or so inputs; bigger gates are built as trees of smaller ones — which is itself multilevel logic. So “two-level” is an idealisation.

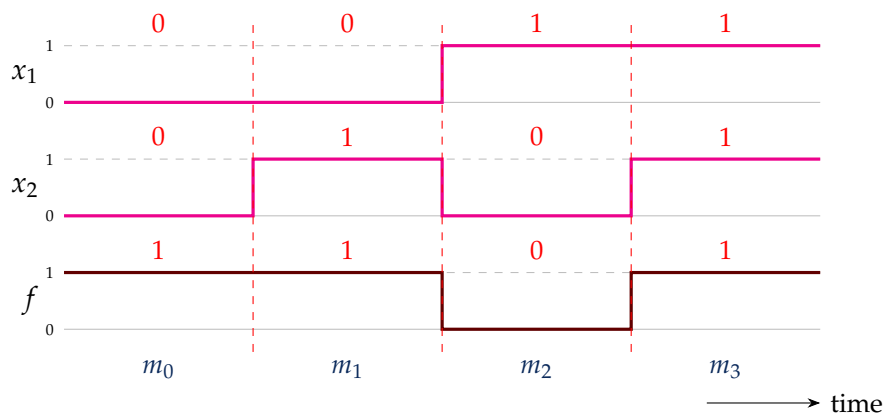
Aside: Timing Diagrams

From the slides:

Aside: Timing Diagrams Timing diagrams focus on conditions changing within and among lifelines along a linear time axis. Waveforms x_1, x_2, f against time; $f(x_1, x_2) = ?$ (Handwritten: interval values $x_1 = 0, 0, 1, 1$; $x_2 = 0, 1, 0, 1$; $f = 1, 1, 0, 1$; truth table rows $m_0..m_3$ with $f = 1, 1, 0, 1$; $\sum m(0, 1, 3)$; $\bar{x}_1 \bar{x}_2 + \bar{x}_1 x_2 + x_1 x_2 \Rightarrow \bar{x}_1(\bar{x}_2 + x_2) + x_1 x_2 \Rightarrow \bar{x}_1 + x_1 x_2 = \bar{x}_1 + x_2$. Also: “don’t care”, “0/1 $\rightarrow$ X”, and $\bar{x}_1 \bar{x}_2 = \bar{x}_1 + x_2$.)

7.1 Reading a timing diagram

A timing diagram plots each signal (a “lifeline”) against a common time axis. For a combinational circuit, read it by slicing time into intervals where no input changes; within each interval the inputs form one row of the truth table and the output shows the function’s value for that row.



interval	x_1	x_2	f
m_0	0	0	1
m_1	0	1	1
m_2	1	0	0
m_3	1	1	1

$$f(x_1, x_2) = \sum m(0, 1, 3) = \prod M(2)$$

$$\begin{aligned}
 f &= \bar{x}_1 \bar{x}_2 + \bar{x}_1 x_2 + x_1 x_2 \\
 &= \bar{x}_1(\bar{x}_2 + x_2) + x_1 x_2 = \bar{x}_1 + x_1 x_2 \\
 &= \bar{x}_1 + x_2 \quad (\text{T11D})
 \end{aligned}$$

POS route, even faster: $f = M_2 = \bar{x}_1 + x_2$.

The only 0 is at $x_1 = 1, x_2 = 0$, so a single maxterm describes f : this is the textbook case where POS is shorter than SOP. Equivalently $f = \overline{x_1 x_2}$ — a NAND of x_1 and x_2 (handwritten).

Beyond the Slides : “if x_1 then x_2 ”

$\bar{x}_1 + x_2$ is logical *implication*, $x_1 \Rightarrow x_2$: it is false only when the premise x_1 is true and the conclusion x_2 is false.

7.2 Don't-care conditions

The handwritten note “don't care: $0/1 \rightarrow X$ ” introduces an idea used heavily in Week 4. Sometimes an input combination can never occur, or its output is irrelevant. Its output is then marked X (or d) in the truth table, and the designer may pick 0 or 1, whichever gives the simpler circuit.

Worked Example

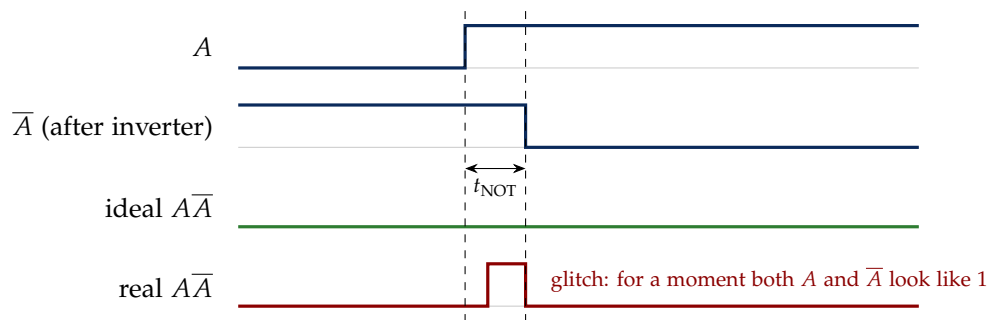
Suppose in the diagram above interval m_2 never occurs (its output is X). Then:

- choose $X = 0$: $f = \overline{x_1} + x_2$ (as before);
- choose $X = 1$: $f = 1$ for all rows — a wire tied high, no gates at all.

The second choice is cheaper. Writing $f = \sum m(0, 1, 3) + d(2)$ records the freedom.

7.3 Beyond the slides: what real timing diagrams show

The slide's diagram is idealised: f changes at the same instant as the inputs. Real gates have a propagation delay t_{pd} , so f lags its inputs. Worse, when two paths through a circuit have different delays, the output can briefly take a wrong value — a **glitch**.



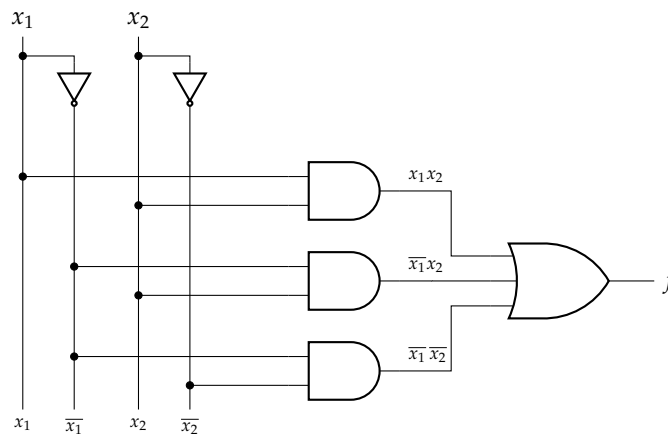
Algebraically $A\overline{A} = 0$ always, but the inverter's delay means $\overline{A}$ is still 1 for a short while after A rises. This is why the functional specification (Boolean algebra) is not the whole story and why the slides in Week 2 insisted on a separate timing specification. Sequential circuits (Module 3) sample outputs only after they have settled, which hides such glitches.

Tutorial and Practice Problems from the Slides

8.1 Tutorial 1: optimise this circuit

From the slides:

Tutorial Problem Somebody created the following circuit for $f(x_1, x_2)$. Can you optimize it further? Circuit (A): inverters produce $\bar{x}_1$ and $\bar{x}_2$; three AND gates compute x_1x_2 , $\bar{x}_1x_2$, $\bar{x}_1\bar{x}_2$; an OR combines them into f . Handwritten: $x_1x_2 + \bar{x}_1x_2 + \bar{x}_1\bar{x}_2 \Rightarrow \bar{x}_1 + x_1x_2 \Rightarrow \bar{x}_1 + x_2$; “room for optimization”; (B) a single gate on x_1 and inverted x_2 with an inverted output; (C) an OR gate on $\bar{x}_1$ and x_2 .



(A) original: 2 NOT, 3 AND2, 1 OR3

8.1.1 Solution

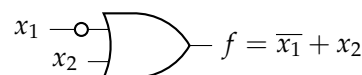
Step 1: function. $f = x_1x_2 + \bar{x}_1x_2 + \bar{x}_1\bar{x}_2 = \sum m(0, 1, 3)$ — the same function as the timing-diagram example, so the result is the same:

$$f = \bar{x}_1(x_2 + \bar{x}_2) + x_1x_2 = \bar{x}_1 + x_1x_2 = \bar{x}_1 + x_2.$$

Step 2: circuits.



(B) NAND with x_2 inverted



(C) OR with x_1 inverted

(B) and (C) are the same gate by De Morgan: $\overline{x_1x_2} = \bar{x}_1 + \bar{x}_2 = \bar{x}_1 + x_2$. Either needs one inverter and one 2-input gate.

	gates (excl. NOT)	inverters	gate inputs
(A) original	4	2	9
(B)/(C) optimised	1	1	2

Key Idea

Always ask: which rows are 0? Here there is only one ($x_1 = 1, x_2 = 0$), so f is a single maxterm $\overline{x_1} + x_2$. A circuit with “room for optimization” is usually a canonical form built without simplification.

8.2 Tutorial 2: the three-switch bulb

From the slides:

Tutorial Problem / Practice Problem A room with three doors; switches X, Y, Z are located by the doors. The bulb in the room needs to be made OFF or ON by toggling *any one of the three switches*. Implement a circuit controlling the bulb. Practice steps (handwritten): (1) draw the truth table for this problem; (2) implement the circuit; (3) try physically implementing the logical circuit.

8.2.1 Step 1: the specification

Let $X, Y, Z \in \{0, 1\}$ be the switch positions and L the bulb (1 = ON). Requirement: *flipping any one switch flips the bulb*. We also need a starting point; assume all switches down means OFF: $L(0, 0, 0) = 0$.

Deriving the table. Flipping one switch moves to a row that differs in exactly one bit, and L must change. Starting from $L(000) = 0$, any row with k switches up is reachable by k single flips, each flipping L . So $L = 1$ exactly when k is odd:

row	X	Y	Z	L	switches up
0	0	0	0	0	0 (even)
1	0	0	1	1	1
2	0	1	0	1	1
3	0	1	1	0	2
4	1	0	0	1	1
5	1	0	1	0	2
6	1	1	0	0	2
7	1	1	1	1	3 (odd)

L is the **odd-parity** function.

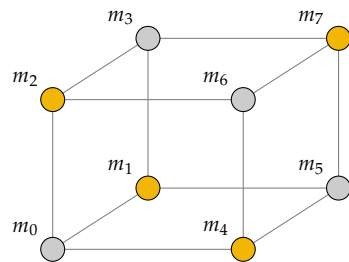
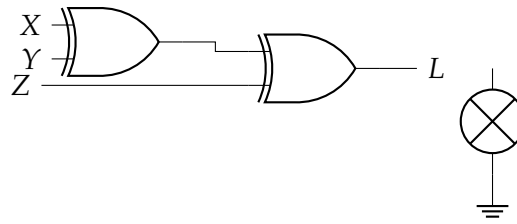
8.2.2 Step 2: the logic

$$L(X, Y, Z) = \sum m(1, 2, 4, 7) = \overline{X}\overline{Y}Z + \overline{X}Y\overline{Z} + X\overline{Y}\overline{Z} + XYZ.$$

No two of these minterms differ in only one variable (they are the non-adjacent corners of the cube), so *no pair combines*: the canonical SOP is already minimal as an SOP (4 AND3 + OR4). But the function has a much neater multilevel form using XOR ($A \oplus B = A\overline{B} + \overline{A}B$, 1 when inputs differ):

$$L = X \oplus Y \oplus Z.$$

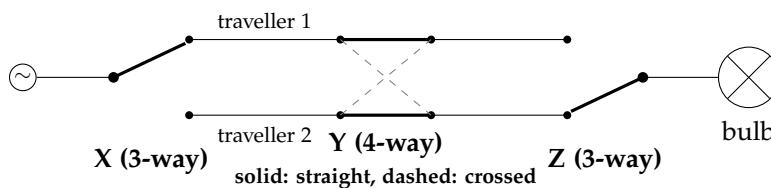
Proof. $X \oplus Y$ is 1 iff an odd number of X, Y is 1. XORing with Z flips the result iff $Z = 1$, so $(X \oplus Y) \oplus Z$ is 1 iff an odd number of all three is 1 — the table above.



yellow = bulb ON
every edge (one switch flipped)
joins ON to OFF $\Rightarrow$ no
two ON corners are adjacent,
so SOP cannot be simplified

8.2.3 Step 3: physical implementation

Home wiring does this without any logic gates, using two **3-way** (single-pole double-throw) switches at the ends and one **4-way** (crossover) switch in the middle, joined by two “traveller” wires.



Each switch either keeps or swaps which traveller carries the live connection. The bulb lights when X 's chosen traveller arrives at the traveller Z is connected to. Flipping any one switch changes that match, so it toggles the bulb: the wiring physically computes $X \oplus Y \oplus Z$. With more doors, add more 4-way switches in the middle.

Beyond the Slides : XOR from basic gates

$A \oplus B = A\bar{B} + \bar{A}B$ (SOP of rows 01 and 10). With NANDs only it takes 4 gates: $t = \overline{AB}$, then $A \oplus B = \overline{At} \cdot \overline{Bt}$. XOR appears again in Week 4 as the heart of the equality comparator and the adder.

Part II

Practice

Chapter 9

Numerical Problems with Worked Solutions

Unless stated otherwise, variable order is alphabetical with the first variable as MSB.

9.1 Minterm and maxterm notation

Problem 1 *Row indices*

Find the minterm number of (a) $ABC = 110$, (b) $ABCD = 1011$, (c) $ABCD = 0101$.

Solution

(a) $4 + 2 + 0 = 6$. (b) $8 + 0 + 2 + 1 = 11$. (c) $0 + 4 + 0 + 1 = 5$.

Problem 2 *Writing m_i and M_i*

Write (a) m_5 and M_5 for $F(A, B, C)$; (b) m_9 and M_9 for $F(A, B, C, D)$.

Solution

(a) $5 = 101$: $m_5 = A\bar{B}C$ (1 plain, 0 barred); $M_5 = \bar{A} + B + \bar{C}$ (0 plain, 1 barred). (b) $9 = 1001$: $m_9 = A\bar{B}\bar{C}D$; $M_9 = \bar{A} + B + C + \bar{D}$. Check: $M_i = \bar{m}_i$ by De Morgan.

Problem 3 *Order matters*

$F(A, B, C) = \sum m(1)$. Rewrite F with variable order (C, B, A) .

Solution

$m_1 = \bar{A}\bar{B}C$: $A = 0, B = 0, C = 1$. In order C, B, A the bit pattern is $100 = 4$. So $F(C, B, A) = \sum m(4)$.

Problem 4 *How many terms?*

A 4-variable function is 1 in 11 rows. How many minterms in its canonical SOP and how many maxterms in its canonical POS?

Solution

SOP: 11 minterms. POS: one maxterm per 0: $16 - 11 = 5$ maxterms. (Always $\#m + \#M = 2^n$.)

Problem 5 *Choosing SOP or POS*

A 4-input function has 13 ones. Compare the canonical SOP and POS costs (gates and gate inputs).

Solution

SOP: 13 AND4 + one OR13: $52 + 13 = 65$ inputs. POS: 3 OR4 + one AND3: $12 + 3 = 15$ inputs. POS is over four times cheaper — use it when 0s are few.

9.2 Conversions

Problem 6 *Minterm to maxterm*

$F(A, B, C) = \sum m(0, 2, 3, 7)$. Write F as $\prod M$, and $\bar{F}$ as both $\sum m$ and $\prod M$.

Solution

All indices: 0–7. Missing: $\{1, 4, 5, 6\}$. $F = \prod M(1, 4, 5, 6)$ (rule 1). $\bar{F} = \sum m(1, 4, 5, 6)$ (rule 3) $= \prod M(0, 2, 3, 7)$ (rule 4).

Problem 7 *Four variables*

$F(A, B, C, D) = \sum m(1, 4, 7, 9, 12, 15)$. Give F in $\prod M$ form and $\bar{F}$ in $\sum m$ form.

Solution

Missing from 0–15: $\{0, 2, 3, 5, 6, 8, 10, 11, 13, 14\}$. $F = \prod M(0, 2, 3, 5, 6, 8, 10, 11, 13, 14)$; $\bar{F} = \sum m(0, 2, 3, 5, 6, 8, 10, 11, 13, 14)$.

Problem 8 *Complement lists*

$F(A, B, C) = \sum m(0, 1, 2, 5)$. Find $\bar{F}$ as $\sum m$ and as $\prod M$.

Solution

$\bar{F} = \sum m(3, 4, 6, 7) = \prod M(0, 1, 2, 5)$.

Problem 9 *SOP and POS agree*

For $F(A, B) = \sum m(1, 2)$, write $\prod M$ form and multiply it out to confirm it matches.

Solution

$F = \prod M(0, 3) = (A + B)(\bar{A} + \bar{B})$. Expand: $A\bar{A} + A\bar{B} + \bar{A}B + B\bar{B} = A\bar{B} + \bar{A}B = \sum m(2, 1)$ ✓ (this is XOR).

9.3 Minimisation

Problem 10 *Odd rows*

Minimise $F(A, B, C) = \sum m(1, 3, 5, 7)$.

Solution

Every row with $C = 1$. Pairwise: $m_1 + m_3 = \bar{A}C$, $m_5 + m_7 = AC$, then $\bar{A}C + AC = C$. $F = C$.

Problem 11 *Same function via POS*

Write F of the previous problem as $\prod M$ and minimise.

Solution

$F = \prod M(0, 2, 4, 6)$: $(A + B + C)(A + \bar{B} + C)(\bar{A} + B + C)(\bar{A} + \bar{B} + C)$. Pair by T9D: first two give $A + C$, last two give $\bar{A} + C$; then $(A + C)(\bar{A} + C) = C$. Same answer, as it must be.

Problem 12 *Even rows*

Minimise $F(A, B, C) = \sum m(0, 2, 4, 6)$.

Solution

All rows with $C = 0$: $F = \bar{C}$.

Problem 13 *Five minterms*

Minimise $F(A, B, C) = \sum m(1, 3, 5, 6, 7)$.

Solution

m_1, m_3, m_5, m_7 are the rows with $C = 1$: they give C . $m_6 = ABC$ remains; pair it with m_7 (reuse, T3): $ABC + ABC = AB$. $F = C + AB$. Check m_6 : $0 + 1 = 1 \checkmark$; $m_4 = 100$: $0 + 0 = 0 \checkmark$.

Problem 14 *Lower half plus one*

Minimise $F(A, B, C) = \sum m(0, 1, 2, 3, 7)$.

Solution

m_0 – m_3 are the rows with $A = 0$: $\bar{A}$. $m_7 = ABC$ pairs with $m_3 = \bar{A}BC$ to give BC . $F = \bar{A} + BC$.

Problem 15 *Four corners*

Minimise $F(A, B, C, D) = \sum m(0, 2, 8, 10)$.

Solution

Binary: 0000, 0010, 1000, 1010. $B = 0$ and $D = 0$ in all four; A, C take all combinations. $F = \bar{B}\bar{D}$. Algebraically: $m_0 + m_2 = \bar{A}\bar{B}\bar{D}$, $m_8 + m_{10} = A\bar{B}\bar{D}$, sum = $\bar{B}\bar{D}$.

Problem 16 *The lecture's two-level example in POS*

$Y(A, B, C) = \sum m(0, 4, 5)$. Find its canonical POS and minimise it.

Solution

$Y = \prod M(1, 2, 3, 6, 7)$. Pair (T9D, reusing maxterms): $M_2M_3 = (A + \bar{B} + C)(A + \bar{B} + \bar{C}) = A + \bar{B}$; $M_6M_7 = \bar{A} + \bar{B}$; then $(A + \bar{B})(\bar{A} + \bar{B}) = \bar{B}$. $M_1M_3 = (A + B + \bar{C})(A + \bar{B} + \bar{C}) = A + \bar{C}$. $Y = \bar{B}(A + \bar{C})$ — same as the SOP route in Chapter 2.

Problem 17 *Don't-cares*

$F(A, B, C) = \sum m(1, 3) + d(5, 7)$. Find the simplest F .

Solution

Choose both don't-cares as 1: ones become $\{1, 3, 5, 7\} \Rightarrow F = C$. (Choosing them 0 gives $\overline{A}C$, which needs an extra literal.)

9.4 Universal gates**Problem 18** *Gate counts*

How many 2-input NAND gates are needed for (a) NOT, (b) AND, (c) OR, (d) XOR?

Solution

(a) 1 (tie inputs). (b) 2 (NAND + NAND-inverter). (c) 3 (two inverters + NAND). (d) 4 ($t = \overline{A}\overline{B}$, $\overline{A}t$, $\overline{B}t$, then NAND of those).

Problem 19 *NAND–NAND*

Implement $F = AB + CD$ with NANDs only, and evaluate at $A = 0, B = 1, C = 1, D = 1$.

Solution

$F = \text{NAND}(\text{NAND}(A, B), \text{NAND}(C, D))$: 3 gates. $\text{NAND}(0, 1) = 1$, $\text{NAND}(1, 1) = 0$, $F = \text{NAND}(1, 0) = 1$. Direct: $0 + 1 = 1 \checkmark$.

Problem 20 *A complemented literal*

Implement $F = \overline{A}\overline{B} + C$ with NANDs only. How many gates?

Solution

$F = \overline{\overline{A}\overline{B} \cdot \overline{C}}$. Need $\overline{B} = \text{NAND}(B, B)$ and $\overline{C} = \text{NAND}(C, C)$; then $\text{NAND}(A, \overline{B})$; then the output NAND. Total: 4 NANDs.

Problem 21 *NOR–NOR*

Implement $F = (A + B)(C + D)$ with NORs only; evaluate at $A = B = 0, C = 1, D = 0$.

Solution

$F = \text{NOR}(\text{NOR}(A, B), \text{NOR}(C, D))$. $\text{NOR}(0, 0) = 1$, $\text{NOR}(1, 0) = 0$, $F = \text{NOR}(1, 0) = 0$. Direct: $(0)(1) = 0 \checkmark$.

Problem 22 *AND from NOR*

Build AB from 2-input NORs and verify for all inputs.

Solution

$AB = \overline{\overline{A} + \overline{B}} = \text{NOR}(\text{NOR}(A, A), \text{NOR}(B, B))$: 3 gates. Rows: $00 \rightarrow \text{NOR}(1, 1) = 0$; $01 \rightarrow \text{NOR}(1, 0) = 0$; $10 \rightarrow \text{NOR}(0, 1) = 0$; $11 \rightarrow \text{NOR}(0, 0) = 1 \checkmark$.

9.5 Multilevel logic and timing**Problem 23** *Factoring*

$F = AC + AD + BC + BD$. Factor it and compare gate inputs.

Solution

$F = A(C + D) + B(C + D) = (A + B)(C + D)$. SOP: 4 AND2 + OR4 = 12 inputs. Factored: 2 OR2 + AND2 = 6 inputs. Levels: both 2.

Problem 24 *SOP and POS of a multilevel expression*

Write $Y = (A + B)C + D$ in two-level SOP and two-level POS.

Solution

SOP: $AC + BC + D$. POS: write $Y = D + (A + B) \cdot C$ and use $X + PQ = (X + P)(X + Q)$ with $X = D$, $P = A + B$, $Q = C$: $Y = (A + B + D)(C + D)$.

Problem 25 *Critical path*

Gates have delay $\delta = 20$ ps and inverters 10 ps. Find the worst delay of (a) the lecture's multilevel circuit $(A + B)(\overline{C} + D) + \overline{E}$; (b) its two-level SOP $A\overline{C} + AD + B\overline{C} + BD + \overline{E}$.

Solution

(a) Path $C \rightarrow \text{NOT} \rightarrow \text{OR} \rightarrow \text{AND} \rightarrow \text{OR}$: $10 + 3 \times 20 = 70$ ps. (b) $C \rightarrow \text{NOT} \rightarrow \text{AND} \rightarrow \text{OR}$: $10 + 2 \times 20 = 50$ ps. Multilevel saves gates, costs 20 ps here.

Problem 26 *Waveform to function*

In four successive intervals, $(x_1, x_2) = (0, 0), (0, 1), (1, 0), (1, 1)$ and $f = 0, 1, 1, 0$. Find f .

Solution

$f = \sum m(1, 2) = \overline{x_1}x_2 + x_1\overline{x_2} = x_1 \oplus x_2$ (XOR). No simplification is possible in SOP.

Problem 27 *Three-input waveform*

Over 8 intervals, ABC counts $000, 001, \dots, 111$ and f reads $0, 0, 0, 1, 0, 1, 1, 1$. Find the minimal f .

Solution

$f = \sum m(3, 5, 6, 7)$ — the majority function. Pair each with m_7 : $m_3 + m_7 = BC$, $m_5 + m_7 = AC$, $m_6 + m_7 = AB$. $f = AB + BC + AC$.

Problem 28 *Glitch width*

An inverter has delay 30 ps. For $Y = A \cdot \overline{A}$ built with an inverter and an AND (delay 20 ps), estimate the glitch when A rises from 0 to 1.

Solution

For the 30 ps before $\overline{A}$ falls, both AND inputs are 1. The AND output rises 20 ps after A rises and falls 20 ps after $\overline{A}$ falls, so the glitch is about 30 ps wide, starting 20 ps after the edge. (A real gate may filter pulses shorter than its own delay.)

9.6 The switch problem**Problem 29** *Four switches*

Four switches control one bulb; toggling any one toggles the bulb, and all-off means OFF. How many minterms in its canonical SOP, and how many 2-input XOR gates in an XOR implementation?

Solution

$L = W \oplus X \oplus Y \oplus Z$: 1 when an odd number are on. Rows with 1 or 3 ones: $\binom{4}{1} + \binom{4}{3} = 4 + 4 = 8$ minterms (8 AND4 + OR8). XOR chain: 3 gates.

Problem 30 *Why no simplification*

Show that no two minterms of $X \oplus Y \oplus Z$ can be combined by T9.

Solution

T9 combines two minterms that differ in exactly one variable. Flipping one variable changes the number of 1s by one, changing its parity, so the neighbour of an odd-parity row is always an even-parity row (a 0). No two 1s are neighbours.

Subjective Questions with Answers

Q1. What does the notation $F(A, B, C) = \sum m(3, 4, 5, 6, 7)$ mean, and what two things must be agreed for it to be meaningful?

Answer

F is the OR of minterms m_3, m_4, m_5, m_6, m_7 — it is 1 exactly in truth-table rows 3–7 and 0 elsewhere. The number of variables and their order (which is the MSB) must be fixed, since the row number is the binary value of the input pattern.

Q2. Explain, with the lecture example, why “canonical form $\neq$ minimal form”.

Answer

$\sum m(3, 4, 5, 6, 7)$ has five 3-literal minterms (5 AND3 + OR5), but algebra reduces it to $A + BC$ (one AND2 + one OR2). The canonical form is unique and mechanical, one term per truth-table 1, so it includes redundancy; the minimal form removes variables that don’t matter (e.g. C in $A\bar{B}\bar{C} + A\bar{B}C$).

Q3. What is two-level logic? Give its main advantage and main drawback.

Answer

A circuit where every input passes through at most two gate levels (excluding input inverters): AND then OR (SOP) or OR then AND (POS). Advantage: small, predictable delay regardless of the function. Drawback: may need many gates and large fan-in, since it cannot share sub-expressions.

Q4. Explain the POS form and why each sum term “represents one of the zeros” of the function.

Answer

POS writes F as an AND of maxterms, one for each row where $F = 0$. A maxterm is an OR that is 0 only when every literal is 0, i.e. only in its own row. In that row it forces the AND to 0; in every other row it is 1 and does not affect the AND.

Q5. Show that POS of F is the De Morgan of the SOP of $\bar{F}$.

Answer

$\bar{F} = \sum_{i \in S'} m_i$ (S' = rows where $F = 0$). Complementing: $F = \overline{\sum m_i} = \prod \bar{m}_i = \prod M_i$, using De Morgan and $\bar{\bar{m}_i} = M_i$. So $F = \prod M(S')$.

Q6. State the literal rules for writing a minterm and a maxterm from a row, and explain why they are opposite.

Answer

Minterm: 1 $\rightarrow$ plain, 0 $\rightarrow$ complemented (so each literal is 1 in that row and the AND is 1).
 Maxterm: 0 $\rightarrow$ plain, 1 $\rightarrow$ complemented (so each literal is 0 in that row and the OR is 0). One must be 1 exactly on the row, the other 0 exactly on the row, hence the opposite choices.

Q7. The slide warns that $\prod M(0,1,2)$ “is not the complement of the function”. Explain.

Answer

$\prod M(0,1,2)$ equals F itself: it is 0 on rows 0, 1, 2, which are F ’s zeros. The complement is $\bar{F} = \sum m(0,1,2)$ (same indices with minterms) or $\prod M(3,4,5,6,7)$. Students confuse “built from the zeros” with “the complement”.

Q8. State the four conversion rules between $\sum m$ and $\prod M$ and give the unifying principle.

Answer

(1) $\sum m(S) = \prod M(S')$; (2) $\prod M(S') = \sum m(S)$; (3) $F = \sum m(S) \Rightarrow \bar{F} = \sum m(S')$; (4) $F = \sum m(S) \Rightarrow \bar{F} = \prod M(S)$. Principle: changing both the symbol and the index set keeps the function; changing exactly one complements it.

Q9. When would you prefer POS over SOP?

Answer

When the function has fewer 0s than 1s: canonical POS has one term per 0, SOP one per 1.
 Example: $f = \sum m(0,1,3) = \prod M(2) = \bar{x}_1 + x_2$. In NOR-based technology, POS maps directly to NOR–NOR.

Q10. What does it mean for a gate to be universal? Prove that NAND is universal.

Answer

A gate type is universal if any Boolean function can be built using only that gate. {AND, OR, NOT} is universal (SOP). NAND gives NOT ($\overline{AA} = \bar{A}$), AND ($\overline{AB}$), and OR ($\overline{\overline{A}\overline{B}} = A + B$ by De Morgan), so NAND alone suffices.

Q11. Describe the transformation from AND–OR to NAND–NAND and justify it.

Answer

Insert two inverters (bubbles) on each wire between the AND and OR levels; this changes nothing (involution). Absorb one bubble into each AND, making it a NAND; the OR with bubbled inputs is a NAND by De Morgan. Algebraically $F = \sum P_k = \prod \overline{\overline{P_k}}$. So every gate of an SOP circuit is replaced by a NAND with unchanged wiring.

Q12. Why are NAND and NOR preferred to AND and OR in CMOS?

Answer

CMOS gates are naturally inverting: a 2-input NAND or NOR takes 4 transistors, while AND/OR need an extra inverter (6 transistors) and an extra delay. So NAND/NOR are smaller and faster; NAND is often preferred because its series nMOS transistors switch faster than series pMOS.

Q13. Why is multilevel logic often more compact, and what does it cost?

Answer

Factoring shares sub-expressions: $(A + B)(\bar{C} + D)$ computes $A + B$ once instead of wiring A and B into several AND gates. This reduces gates, literals and fan-in. The cost is extra levels on the critical path, each adding a gate delay δ .

Q14. Using $Y = (A + B)(\bar{C} + D) + \bar{E}$, compare multilevel, SOP and POS realisations.

Answer

Multilevel: 4 gates, 8 inputs, 3 levels. SOP $A\bar{C} + AD + B\bar{C} + BD + \bar{E}$: 5 gates, 13 inputs, 2 levels. POS $(A + B + \bar{E})(\bar{C} + D + \bar{E})$: 3 gates, 8 inputs, 2 levels. Multilevel beats SOP on size but is slower; here POS happens to be best on both.

Q15. How do you extract a Boolean function from a timing diagram?

Answer

Split time into intervals where the inputs are constant. Each interval gives a row of the truth table (input values and the output value). Collect the rows, then write $\sum m$ or $\prod M$ and minimise. For the slide: rows $00 \rightarrow 1, 01 \rightarrow 1, 10 \rightarrow 0, 11 \rightarrow 1$, so $f = \sum m(0,1,3) = \bar{x}_1 + x_2$.

Q16. What are don't-care conditions and how are they used?

Answer

Input combinations whose output doesn't matter (impossible or ignored), marked X or d . The designer may choose each to be 0 or 1, whichever gives a simpler expression. E.g. $\sum m(1,3) + d(5,7)$ becomes C by choosing both don't-cares as 1.

Q17. Why is the functional specification alone insufficient? Explain a glitch.

Answer

Gates have delays. If a signal reaches a gate by two paths with different delays, the output can briefly take a wrong value. For $A \cdot \bar{A}$ (always 0 in algebra), after A rises the inverter output stays 1 for its delay, so the AND briefly outputs 1. Timing specifications describe such behaviour.

Q18. Derive the logic for the three-switch bulb and explain why its SOP cannot be simplified.

Answer

Toggling any switch must toggle the bulb; with all-off = OFF, the bulb is ON exactly when an odd number of switches are on: $L = \sum m(1, 2, 4, 7) = X \oplus Y \oplus Z$. Rows differing in one switch have opposite parity, so no two 1-rows are adjacent and T9 can never combine minterms. The XOR form (2 gates) is the compact realisation.

Q19. How is the three-switch function implemented physically, without logic gates?

Answer

Two 3-way (SPDT) switches at the ends and a 4-way (crossover) switch in the middle, connected by two traveller wires. Each switch either keeps or swaps which traveller is live; the bulb lights when the live traveller matches the one the last switch selects. Flipping any switch changes that match, so the wiring computes the XOR of the three positions.

Q20. Summarise the design flow from a truth table to a NAND-only circuit.

Answer

(1) Write the truth table (or read it from a timing diagram). (2) Write canonical SOP ($\sum m$) or POS ($\prod M$), using don't-cares if available. (3) Minimise with Boolean algebra (combine neighbours, absorption, reuse terms). (4) Optionally factor into multilevel form if size matters more than delay. (5) Convert SOP to NAND–NAND (or POS to NOR–NOR).

Chapter 11

Multiple-Choice Questions

Answers and explanations follow in Section 11.2.

11.1 Questions

Minterm notation and canonical SOP

1. For $F(A, B, C)$, the row $ABC = 101$ is minterm:
 - (a) m_3
 - (b) m_5
 - (c) m_6
 - (d) m_{101}
2. m_6 of $F(A, B, C)$ is:
 - (a) ABC
 - (b) $\overline{A}\overline{B}C$
 - (c) $\overline{A} + \overline{B} + C$
 - (d) $A\overline{B}C$
3. $\sum m(3, 4, 5, 6, 7)$ over A, B, C minimises to:
 - (a) $A + BC$
 - (b) $AB + C$
 - (c) $A + B + C$
 - (d) $\overline{A} + BC$
4. Which statement is TRUE?
 - (a) canonical form is always minimal
 - (b) canonical form is unique for each function
 - (c) minimal form is unique for each function
 - (d) SOP always has fewer terms than POS
5. A canonical SOP for n inputs uses AND gates with how many inputs each?
 - (a) 2
 - (b) n
 - (c) 2^n
 - (d) depends on the function
6. $F(A, B, C) = \sum m(1)$; with order (C, B, A) this is:
 - (a) $\sum m(1)$

- (b) $\sum m(2)$
- (c) $\sum m(4)$
- (d) $\sum m(6)$

7. $F(A, B, C) = \sum m(1, 3, 5, 7)$ equals:

- (a) A
- (b) $\bar{C}$
- (c) C
- (d) $B + C$

8. Two minterms can be combined by $XY + X\bar{Y} = X$ when they:

- (a) differ in exactly one variable
- (b) differ in two variables
- (c) are complements
- (d) share no variables

Two-level and POS

9. SOP leads to which circuit structure?

- (a) OR-AND
- (b) AND-OR
- (c) NAND-NOR
- (d) single gate

10. The maxterm for row $ABC = 010$ is:

- (a) $\bar{A}\bar{B}\bar{C}$
- (b) $A + \bar{B} + C$
- (c) $\bar{A} + B + \bar{C}$
- (d) $A\bar{B}C$

11. A maxterm is 0 in how many rows?

- (a) 0
- (b) 1
- (c) n
- (d) $2^n - 1$

12. POS is also called:

- (a) disjunctive normal form
- (b) minterm expansion
- (c) conjunctive normal form
- (d) sum of products

13. $F(A, B, C) = \sum m(3, 4, 5, 6, 7)$ in maxterm form is:

- (a) $\prod M(3, 4, 5, 6, 7)$

- (b) $\prod M(0,1,2)$
- (c) $\sum m(0,1,2)$
- (d) $\prod m(0,1,2)$

14. In the POS rule, a 1 in an input column gives:

- (a) a plain literal
- (b) a complemented literal
- (c) no literal
- (d) a constant 1

15. For input 010, $F = (A + B + C)(A + B + \overline{C})(A + \overline{B} + C)$ equals:

- (a) 1
- (b) 0
- (c) undefined
- (d) depends on wiring

16. $\prod M(0,1,2)$ for $F(A,B,C)$ is:

- (a) $\overline{F}$
- (b) F
- (c) F only if minimal
- (d) neither

17. $(A + B + C)(A + B + \overline{C})$ simplifies to:

- (a) $A + B$
- (b) C
- (c) AB
- (d) 1

18. M_i and m_i are related by:

- (a) $M_i = m_i$
- (b) $M_i = \overline{m_i}$
- (c) $M_i = m_{7-i}$
- (d) $M_i = m_i + 1$

Conversions

19. If $F = \sum m(3,4,5,6,7)$, then $\overline{F} =$

- (a) $\sum m(3,4,5,6,7)$
- (b) $\prod M(0,1,2)$
- (c) $\sum m(0,1,2)$
- (d) $\prod M(0,1,2,3)$

20. If $F = \sum m(3,4,5,6,7)$, then $\overline{F}$ in maxterm form is:

- (a) $\prod M(3,4,5,6,7)$

- (b) $\prod M(0, 1, 2)$
- (c) $\sum m(3, 4, 5, 6, 7)$
- (d) $\prod M(0, \dots, 7)$

21. For 4 variables, $\sum m(0, 5)$ as $\prod M$ has how many maxterms?

- (a) 2
- (b) 6
- (c) 14
- (d) 16

22. Keeping the function while changing $\sum \rightarrow \prod$ requires:

- (a) same indices
- (b) complementary indices
- (c) reversed indices
- (d) doubled indices

Universal gates

23. A universal gate is one that:

- (a) is used in every chip
- (b) can implement any Boolean function alone
- (c) has unlimited inputs
- (d) has no delay

24. How many 2-input NANDs make a 2-input OR?

- (a) 1
- (b) 2
- (c) 3
- (d) 4

25. How many 2-input NORs make a 2-input AND?

- (a) 1
- (b) 2
- (c) 3
- (d) 4

26. An SOP maps directly to:

- (a) NOR–NOR
- (b) NAND–NAND
- (c) XOR–XOR
- (d) NAND–NOR

27. A POS maps directly to:

- (a) NOR–NOR

- (b) NAND–NAND
- (c) AND–OR
- (d) XNOR–XNOR

28. $\text{NAND}(A, A)$ equals:

- (a) A
- (b) $\overline{A}$
- (c) 0
- (d) 1

29. NAND–NAND for $AB + CD$ is:

- (a) $\text{NAND}(\text{NAND}(A, B), \text{NAND}(C, D))$
- (b) $\text{NAND}(A, B, C, D)$
- (c) $\text{NOR}(\text{NAND}(A, B), \text{NAND}(C, D))$
- (d) $\text{NAND}(\text{NOR}(A, B), \text{NOR}(C, D))$

30. In CMOS, a 2-input NAND needs how many transistors?

- (a) 2
- (b) 4
- (c) 6
- (d) 8

Multilevel logic and timing

31. Multilevel logic is usually:

- (a) faster and larger
- (b) more compact but can be slower
- (c) always minimal
- (d) always two gate delays

32. $(A + B)(\overline{C} + D) + \overline{E}$ expanded to SOP has how many product terms?

- (a) 3
- (b) 4
- (c) 5
- (d) 6

33. $AC + AD + BC + BD$ factors to:

- (a) $(A + B)(C + D)$
- (b) $(A + C)(B + D)$
- (c) $ABCD$
- (d) $(A + D)(B + C)$

34. In the slide's timing diagram, $f(x_1, x_2) =$

- (a) $x_1 + \overline{x_2}$

- (b) $\overline{x_1} + x_2$
- (c) $x_1 x_2$
- (d) $x_1 \oplus x_2$

35. The circuit's delay is set by:

- (a) the shortest path
- (b) the critical (slowest) path
- (c) the number of inputs
- (d) the number of outputs

36. A "don't care" output may be set to:

- (a) only 0
- (b) only 1
- (c) 0 or 1, whichever simplifies the circuit
- (d) neither

37. A glitch is:

- (a) a permanent wrong output
- (b) a brief wrong output caused by unequal path delays
- (c) a manufacturing defect
- (d) a don't-care row

Tutorial problems

38. The tutorial circuit $x_1 x_2 + \overline{x_1} x_2 + \overline{x_1} \overline{x_2}$ optimises to:

- (a) x_2
- (b) $\overline{x_1} + x_2$
- (c) $x_1 + \overline{x_2}$
- (d) 1

39. The three-switch bulb (all-off = OFF) is:

- (a) $X + Y + Z$
- (b) XYZ
- (c) $X \oplus Y \oplus Z$
- (d) majority

40. How many 1s in the truth table of $X \oplus Y \oplus Z$?

- (a) 1
- (b) 3
- (c) 4
- (d) 7

11.2 Answers and Explanations

1 b	2 a	3 a	4 b	5 b	6 c	7 c	8 a	9 b	10 b
11 b	12 c	13 b	14 b	15 b	16 b	17 a	18 b	19 c	20 a
21 c	22 b	23 b	24 c	25 c	26 b	27 a	28 b	29 a	30 b
31 b	32 c	33 a	34 b	35 b	36 c	37 b	38 b	39 c	40 c

1. **(b)** $4 + 0 + 1 = 5$.
2. **(a)** $6 = 110$: A, B plain, C barred.
3. **(a)** The slide derivation.
4. **(b)** Canonical forms are unique by construction; minimal forms may not be unique.
5. **(b)** Each minterm contains all n variables.
6. **(c)** $\overline{A}BC$ read as $CBA = 100 = 4$.
7. **(c)** Exactly the rows with $C = 1$.
8. **(a)** Neighbours differ in one variable, which drops out.
9. **(b)** ANDs then one OR.
10. **(b)** 0 plain, 1 barred.
11. **(b)** Only in its own row.
12. **(c)** CNF = POS = maxterm expansion.
13. **(b)** Complementary indices with $\overline{}$.
14. **(b)** Opposite of the minterm rule.
15. **(b)** The third factor is $0 + 0 + 0 = 0$.
16. **(b)** It is F , built from F 's zeros.
17. **(a)** T9D.
18. **(b)** De Morgan.
19. **(c)** Rule 3.
20. **(a)** Rule 4: same indices, switch symbol.
21. **(c)** $16 - 2 = 14$.
22. **(b)** Rule 1.
23. **(b)** Definition of universality.
24. **(c)** Two inverters + one NAND.
25. **(c)** Two inverters + one NOR.
26. **(b)** Replace every gate by NAND.
27. **(a)** Replace every gate by NOR.
28. **(b)** $\overline{\overline{A}} = A$.
29. **(a)** $\overline{\overline{AB}} \cdot \overline{\overline{CD}} = AB + CD$.
30. **(b)** 2 series nMOS + 2 parallel pMOS.

31. **(b)** Area–delay trade-off.
32. **(c)** $A\bar{C}, AD, B\bar{C}, BD, \bar{E}$.
33. **(a)** Factor like ordinary algebra.
34. **(b)** Only row 10 is 0.
35. **(b)** Critical path.
36. **(c)** Free choice.
37. **(b)** Transient hazard from path-delay mismatch.
38. **(b)** Same function as the timing diagram.
39. **(c)** Odd parity.
40. **(c)** Rows 1, 2, 4, 7.

Appendix A

Formula Sheet, Glossary and Revision

A.1 Formula sheet

Row index	$i = \sum_k x_k 2^k$ (first variable = MSB)
Minterm m_i	AND of all variables; bit 1 $\rightarrow$ plain, bit 0 $\rightarrow$ barred; 1 only in row i
Maxterm M_i	OR of all variables; bit 0 $\rightarrow$ plain, bit 1 $\rightarrow$ barred; 0 only in row i
Relation	$M_i = \overline{m_i}$
Canonical SOP	$F = \sum m(S)$, S = rows where $F = 1$
Canonical POS	$F = \prod M(S')$, S' = rows where $F = 0$
Term counts	$ S + S' = 2^n$
Conversions	$\sum m(S) = \prod M(S')$; $\overline{F} = \sum m(S') = \prod M(S)$
Combining	$XY + X\overline{Y} = X$; $(X + Y)(X + \overline{Y}) = X$
Distribute OR over AND	$X + YZ = (X + Y)(X + Z)$ (SOP $\rightarrow$ POS)
NAND universal	$\overline{A} = \overline{A\overline{A}}$; $AB = \overline{\overline{A}\overline{B}}$; $A + B = \overline{\overline{A}\overline{B}}$ (1, 2, 3 gates)
NOR universal	$\overline{A} = \overline{A + \overline{A}}$; $A + B = \overline{\overline{A}\overline{B}}$; $AB = \overline{\overline{A} + \overline{B}}$ (1, 2, 3 gates)
SOP $\rightarrow$ NAND–NAND	$\sum P_k = \prod \overline{P_k}$
POS $\rightarrow$ NOR–NOR	$\prod S_k = \sum \overline{S_k}$
Delay	$t_{\text{circuit}} = \max_{\text{paths}} \sum \delta_{\text{gate}}$; two-level $\approx 2\delta$ (+ inverter)
Parity (switches)	$L = X \oplus Y \oplus Z = \sum m(1, 2, 4, 7)$

A.2 Glossary

Canonical SOP / POS Unique OR-of-minterms / AND-of-maxterms form of a function.

Conjunctive normal form (CNF) Product of sums.

Critical path The slowest input-to-output path; sets the circuit delay.

Don't care (X, d) Row whose output may be chosen freely.

Fan-in Number of inputs of a gate.

Glitch (hazard) Brief wrong output caused by unequal path delays.

Maxterm expansion Canonical POS.

Minterm expansion Canonical SOP.

Multilevel logic Circuits with more than two gate levels; usually factored.

Timing diagram Signals plotted against a common time axis.

Two-level logic AND–OR or OR–AND circuit.

Universal gate A gate type from which any function can be built (NAND, NOR).

XOR $A \oplus B = A\overline{B} + \overline{A}B$; 1 when inputs differ.

A.3 Revision checklist

- ☐ Compute minterm numbers; write m_i and M_i for any i .
- ☐ Explain why order and number of variables matter in $\sum m$.
- ☐ Derive $\sum m(3,4,5,6,7) = A + BC$ step by step.
- ☐ Draw a two-level AND–OR circuit from an SOP; count gates and levels.
- ☐ Write canonical POS; check a row by hand; explain “not the complement”.
- ☐ Apply all four conversion rules.
- ☐ Build NOT/AND/OR from NAND and from NOR; convert $SOP \rightarrow NAND\text{--}NAND$, $POS \rightarrow NOR\text{--}NOR$.
- ☐ Compare multilevel vs two-level for size and delay.
- ☐ Read a function off a timing diagram; use don’t-cares; explain glitches.
- ☐ Solve both tutorials: optimise the $f(x_1, x_2)$ circuit; design the three-switch bulb.